

Python for GenAI Backends — Full Tutorial

Part 1: OOP (Object-Oriented Programming)

Why this matters for GenAI work

Every LLM SDK you'll touch (OpenAI, Anthropic, LangChain) is built on classes — clients, message objects, tool schemas, agent state. You don't need to be a Java-style OOP expert, but you need to be fluent enough to read and extend these libraries, and to design your own clean abstractions (e.g., a `RAGPipeline` class, an `Agent` class).

1.1 Classes and objects — the absolute basics

A **class** is a blueprint. An **object** (or instance) is something built from that blueprint.

```
class LLMClient:
    def __init__(self, model_name, api_key):
        self.model_name = model_name    # instance attribute
        self.api_key = api_key

    def generate(self, prompt):
        return f"[{self.model_name}] response to: {prompt}"

# creating objects (instances)
client1 = LLMClient("claude-sonnet-4-6", "key-abc")
client2 = LLMClient("gpt-5", "key-xyz")

print(client1.generate("hello"))    # [claude-sonnet-4-6] response to: hello
print(client2.generate("hello"))    # [gpt-5] response to: hello
```

Things to notice:

- `__init__` runs automatically when you create an object — it's the constructor.
- `self` refers to "this particular object." Every method needs `self` as its first parameter so it can access that object's own data.
- `client1` and `client2` are separate objects with separate `model_name` values, even though they came from the same class.

Try it yourself (Exercise 1.1): Write a class `EmbeddingModel` with attributes `name` and `dimensions`, and a method `embed(text)` that returns a fake vector — just `[0.0] * self.dimensions`. Create two instances with different dimensions and print their embeddings.

1.2 The four OOP pillars — explained practically

Encapsulation

Bundling data + behavior together, and controlling access to internals.

```
class VectorStore:
    def __init__(self):
        self._documents = []    # underscore = "internal, don't touch directly" (c

    def add_document(self, doc):
        self._documents.append(doc)

    def count(self):
        return len(self._documents)
```

Python doesn't have true `private` like Java. A single underscore `_name` is a convention meaning "internal." A double underscore `__name` triggers **name mangling** (makes it harder to access from outside, not impossible) — used rarely, mostly to avoid accidental overrides in subclasses.

Inheritance

A class can reuse and extend another class.

```
class BaseRetriever:
    def __init__(self, top_k=5):
        self.top_k = top_k

    def retrieve(self, query):
        raise NotImplementedError("Subclasses must implement this")

class VectorRetriever(BaseRetriever):
    def __init__(self, top_k=5, vector_db=None):
        super().__init__(top_k)    # call the parent's __init__
        self.vector_db = vector_db
```

```

def retrieve(self, query):
    return self.vector_db.search(query, k=self.top_k)

class HybridRetriever(BaseRetriever):
    def retrieve(self, query):
        # combines vector search + keyword search
        return ["doc from hybrid search"]

```

This is **exactly** the pattern you'll see in LangChain, LlamaIndex, etc. — a `BaseX` class defining an interface, and concrete classes implementing it differently. You will design RAG pipelines this way.

`super().__init__(top_k)` calls the parent class's constructor so you don't duplicate code.

Polymorphism

Different classes responding to the same method call, each in their own way.

```

retrievers = [VectorRetriever(vector_db=some_db), HybridRetriever()]

for r in retrievers:
    print(r.retrieve("what is RAG?")) # each behaves differently, same method na

```

This is the power of inheritance — your calling code doesn't need to know *which* retriever it has; it just calls `.retrieve()` and trusts each class to do the right thing.

Abstraction

Hiding complexity behind a simple interface. `BaseRetriever.retrieve()` raising `NotImplementedError` is a basic way to express "subclasses must provide this." Python's formal way to do this is `abc`:

```

from abc import ABC, abstractmethod

class BaseRetriever(ABC):
    @abstractmethod
    def retrieve(self, query):
        ...

# this will now ERROR if you try to instantiate BaseRetriever directly,
# or if a subclass forgets to implement retrieve()

```

Try it yourself (Exercise 1.2): Create an abstract base class `BaseLLM` with an abstract

method `complete(prompt) -> str`. Create two subclasses `MockLLM` (returns a hardcoded string) and `EchoLLM` (returns the prompt back). Confirm that trying to instantiate `BaseLLM()` directly raises an error.

1.3 Dataclasses — the modern, lightweight way

For simple data-holding classes, writing `__init__` by hand is tedious. `@dataclass` generates it for you.

```
from dataclasses import dataclass, field

@dataclass
class ChatMessage:
    role: str
    content: str
    tokens: int = 0 # default value
    metadata: dict = field(default_factory=dict) # mutable defaults need field(d

msg = ChatMessage(role="user", content="Hello!")
print(msg) # ChatMessage(role='user', content='Hello!', tokens=0, metadata={})
```

Why `field(default_factory=dict)` instead of `metadata: dict = {}`? Because a plain `{}` default would be **shared across all instances** (a classic Python gotcha) — `default_factory` creates a fresh dict per instance.

This is the pattern you'll use constantly for things like conversation history, tool call results, retrieved chunks with metadata.

Try it yourself (Exercise 1.3): Create a `@dataclass` called `RetrievedChunk` with fields `text`, `score` (float), `source` (str), and `metadata` (dict, defaulting to empty). Create 3 instances and sort them by `score` descending using `sorted()`.

1.4 Magic / dunder methods (the ones that actually matter)

```
class ConversationHistory:
    def __init__(self):
        self.messages = []

    def __len__(self):
        return len(self.messages)
```

```

def __repr__(self):
    return f"ConversationHistory({len(self.messages)} messages)"

def __getitem__(self, index):
    return self.messages[index]

def add(self, role, content):
    self.messages.append({"role": role, "content": content})

history = ConversationHistory()
history.add("user", "hi")
history.add("assistant", "hello!")

print(len(history))      # uses __len__ -> 2
print(history)           # uses __repr__ -> ConversationHistory(2 messages)
print(history[0])        # uses __getitem__ -> {'role': 'user', 'content': 'hi'}

```

You don't need to memorize every dunder method. Know that `__init__`, `__repr__`, `__len__`, `__getitem__`, and `__eq__` are the common ones, and that they let your custom objects behave like built-in types (printable, sliceable, comparable).

Try it yourself (Exercise 1.4): Add `__iter__` to `ConversationHistory` so you can do `for msg in history: ...` directly. (Hint: the simplest way is `return iter(self.messages)`.)

Quick self-check before moving to Part 2

You should be able to answer these without looking back:

1. What's the difference between a class and an instance?
2. Why does every method need `self`?
3. What does `super().__init__()` actually do?
4. Why use `field(default_factory=dict)` instead of `= {}` in a dataclass?

If any of these feel shaky, re-read that section before continuing — Part 2 (Decorators) builds on functions being “first-class objects,” which connects back to how methods work here.